

Home > User Guide > Getting Started > Examples > Online Serving

Kv Events Subscriber

Source https://github.com/vllm-project/vllm/blob/main/examples/online_serving/kv_events_subscriber.py ↗.

```
# SPDX-License-Identifier: Apache-2.0
# SPDX-FileCopyrightText: Copyright contributors to the vLLM project
from typing import Any

import msgspec
import zmq
from msgspec.msgpack import Decoder

from vllm.v1.core.kv_cache_utils import ExternalBlockHash

#
# Types copied from vllm.distributed.kv_events
#
class EventBatch(msgspec.Struct, array_like=True, omit_defaults=True, gc=False):
    ts: float
    events: list[Any]

class KVCacheEvent(
    msgspec.Struct, array_like=True, omit_defaults=True, gc=False, tag=True
):
    """Base class for all KV cache-related events"""

class BlockStored(KVCacheEvent):
    block_hashes: list[ExternalBlockHash]
    parent_block_hash: ExternalBlockHash | None
    token_ids: list[int]
    block_size: int

    lora_id: int | None
    """Deprecated: use `lora_name` for KV block key hash.
    Retained for backward compatibility.
    """

    medium: str | None
    lora_name: str | None
```

» Ask AI

📄 🔗 latest ▼

```

extra_keys: list[tuple[Any, ...] | None] | None = None
"""Extra keys used in block hash computation, one entry per block in
block_hashes. Each entry contains MM identifiers, LoRA name, cache_salt,
prompt embeddings data, etc. for that specific block.
"""

group_idx: int | None = None

class BlockRemoved(KVCacheEvent):
    block_hashes: list[ExternalBlockHash]
    medium: str | None
    group_idx: int | None = None

class AllBlocksCleared(KVCacheEvent):
    pass

class KVEventBatch(EventBatch):
    events: list[BlockStored | BlockRemoved | AllBlocksCleared]

def process_event(event_batch):
    print(f"Received event batch at {event_batch.ts}:")
    for event in event_batch.events:
        print(f"  - {event}")

def main():
    decoder = Decoder(type=KVEventBatch)
    last_seq = -1

    context = zmq.Context()

    # Set up the main subscription socket
    sub = context.socket(zmq.SUB)
    sub.connect("tcp://localhost:5557")
    topic = "kv-events"
    sub.setsockopt_string(zmq.SUBSCRIBE, topic)

    # Initialize replay socket
    replay = context.socket(zmq.REQ)
    replay.connect("tcp://localhost:5558")
    poller = zmq.Poller()
    poller.register(replay, zmq.POLLIN)

    print("Listening for KV cache events on topic:", topic)

    while True:
        try:
            if sub.poll(50):
                _, seq_bytes, payload = sub.recv_multipart()

```

 Ask AI

  latest ▼

```

seq = int.from_bytes(seq_bytes, "big")

if last_seq >= 0 and seq > last_seq + 1:
    missed = seq - last_seq - 1
    print(
        f"Missed {missed} messages (last: {last_seq}, current:
{seq})"
    )

    replay.send((last_seq + 1).to_bytes(8, "big"))

while poller.poll(timeout=200):
    seq_bytes, replay_payload = replay.recv_multipart()
    if not replay_payload:
        # End of replay marker is sent as an empty frame
        # for the payload
        break

    replay_seq = int.from_bytes(seq_bytes, "big")

    if replay_seq > last_seq:
        event_batch = decoder.decode(replay_payload)
        process_event(event_batch)
        last_seq = replay_seq
        if replay_seq >= seq - 1:
            break

    event_batch = decoder.decode(payload)
    process_event(event_batch)

    # ... do other periodic work or check for shutdown ...

except KeyboardInterrupt:
    print("Interrupted")
    break
except Exception as e:
    print("Error decoding message:", e)

if __name__ == "__main__":
    main()

```

 Ask AI

  latest ▼